Gemma 4 Rubber-Stamps Its Own Failing Patches

Sugam Panthi University of Southern Mississippi

github.com/Vein05/gemma4-paper-track Supplement: `rubber-stamp-supplement.pdf` Kaggle paper track

Abstract

To determine when a patch is complete, many coding agents ask the model that authored the patch. Self-evaluation of a patch can only be beneficial if the model evaluates its own patches as accurately as it evaluates others' patches. We evaluate how well Gemma 4 31B self-evaluates its patches using the SWE-bench Verified dataset. Nine open models evaluate each patch from 0 to 100, representing the likelihood that the patch will pass hidden tests. The judges are not informed as to which model authored the patch and are compared to outside judges from other model families on the same patch.

We find that Gemma places the highest score on 52 of 85 of its own failing patches, while outside judges place the same patches lower. We also find that Gemma gives no additional trust to its passing patches or to the failing patches of other models. Five other models do not exhibit this behavior on their own failing patches. Gemma ties many patches at 100, so a stricter cutoff on its score does not raise the share of patches that pass. The quarter of patches Gemma trusts most passes 61% of the time, compared to 53% without any check. When we use the GLM-4.5 score, this increases to 84%. When we break ties in Gemma's score by the length of its agent run, this increases to 78% without a second model. We find that Gemma 4 rubber-stamps its own mistakes, so the agent built on it needs a second signal to determine which patches to submit.

1 Introduction

At the end of each task, coding agents are asked to make a decision to either submit the patch, reattempt the task, or hand the task off to a human. Many agents make this decision by asking the model that generated the patch whether it is correct. We evaluate this check for the Gemma 4 31B model used by the Gemma 4 Developer Agent Paper Track on Kaggle. Each judge is presented with a problem and one patch, without being told its author, and asked to score the chance that the patch passes hidden tests on a scale of 0 to 100. A model's own patches may simply be more difficult to judge, so we compare the results with outside judges, who are models from other families that are asked to score the same patches.

In the first instance, we study the publicly released Gemma patches for 50 issues, which were generated by four different agent programs (harnesses). For other models, we use a single harness (mini-SWE-agent), making the harness different only for Gemma. To remove this difference, we run Gemma 4 31B in mini-SWE-agent ourselves for the same 50 issues, and also for 193 held-out issues. We randomly sample 200 issues and drop 7 (Section 2).

In Gemma, we find that it tends to assign the highest score to most of its own failing patches, and that outside judges score these patches lower. We refer to this as rubber-stamping (Section 3.1). Previous work has shown that neither Gemma 3 12B nor Gemma 3 27B exhibit self-preference when judging code (Pombal et al., 2026). Pombal et al. (2026) also compare the frequency with which the judge incorrectly indicates that a program passes a unit test when considering its own programs versus other families. They find no difference for either model (Section 5). Gemma ties so many patches at the top that even a stricter cutoff on Gemma's score does not increase the proportion of patches that pass. However, a cutoff on the score of a second model does increase this proportion, and breaking ties based on the length of Gemma's agent run also increases

the proportion that pass (Section 3.2). Further, we find that Gemma ranks its own patches less accurately than outside judges, and that this gap is smaller than in our first look. We argue that this is weaker evidence than rubber-stamping (Section 3.3). Finally, we find that five other models do not exhibit extra trust or this gap on their own patches (Section 3.4). Section 4 describes the steps to run this audit on another agent.

2 Setup

Patch sets. All issues come from SWE-bench Verified (Jimenez et al., 2023). We call the model that wrote a patch its writer. Other writers' patches are public submissions (SWE-bench experiments) made with mini-SWE-agent by GLM-4.5, gpt-oss-120b, Kimi-K2, Llama-4-Maverick, Qwen3-Coder and, on the Mini issues, Devstral, with their official labels. We removed test files from every patch and dropped patches over 30,000 characters.

Patch set	Issues	Gemma's patches	Scored by all nine judges (Gemma's / others')
Mini issues, mixed harness	50 (django, sphinx)	public runs in four harnesses	150 / 183
Mini issues, matched harness	the same 50	ours, mini-SWE-agent	42 / 183
Held-out issues	193, from 11 repositories	ours, mini-SWE-agent	180 / 777

The Mini issues are SWE-bench Verified Mini (MariusHobbhahn). Their mixed-harness Gemma patches come from public third-party runs (n-pelleriti) in SWE-agent (Yang et al., 2024), openclaw, opencode and zeroclaw. The held-out issues are a fixed random sample of 200 SWE-bench Verified issues outside the Mini issues. In our own runs, the 4-bit competition checkpoint of Gemma 4 31B made one attempt per issue in mini-SWE-agent v1.7.0 (SWE-agent), with its default settings and temperature 0. We graded these patches with SWE-bench's grading code and kept an issue only if its reference patch passes in our environment, which dropped 7 of the 200 held-out issues. Gemma solved 98 of the 193 issues we kept.

Judges. Nine models judged every patch: Gemma 4 31B, gpt-oss-20b, Qwen3.5-9B, GLM-4.7-Flash, and the five writers above other than Devstral. Calls went through OpenRouter at temperature 0 with reasoning switched off, except for gpt-oss, which always reasons and ran at low effort. We asked a judge once more, with a larger output limit, when its answer had no score (a re-ask). All analyses use the patches that all nine judges scored, which leaves Gemma's patches on 180 of the 193 held-out issues (Supplement, Sections B and C). As an exploratory check, we also report the held-out results without the patches that needed a re-ask.

Measures. We use four measures, with 95% intervals from 2,000 bootstrap resamples of issues.

- AUROC: the chance that a judge scores a random passing patch above a random failing one (ties count half).
- Self-judging gap: how much a model's AUROC drops from other writers' patches to its own, minus the mean drop of outside judges on the same patches. Section 3.3 shows both drops.
- Leniency: a judge's score on a patch minus the outside judges' mean score on that patch. We report how much higher a model's mean leniency is on its own failing patches than on other writers' failing patches.
- Rubber-stamp rate: the share of a judge's own failing patches that it scores 100.

Plans. We committed a written plan for every test before computing its data. For our own patch sets, we changed two things after the plan but before reading any results. We added the leniency measure, and we raised the re-ask limit from 3,000 to 8,000 tokens because many answers ran out before the score. We mark as exploratory every analysis chosen after we saw a result.

3 Results

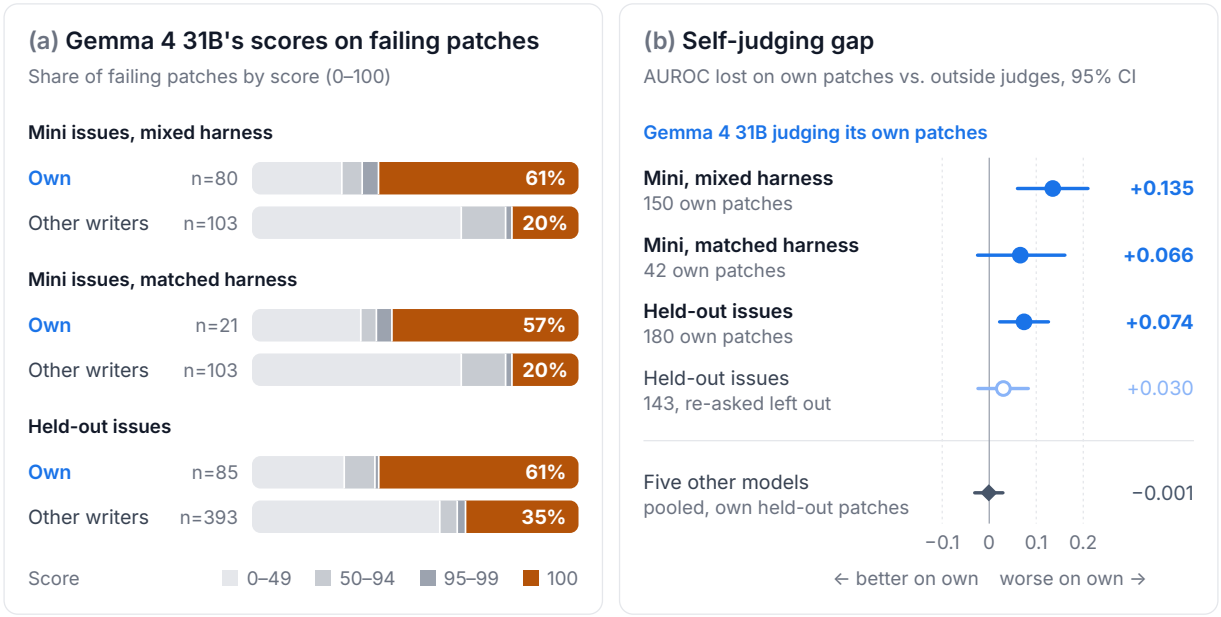


Figure 1. (a) Gemma 4 31B's scores on failing patches, its own and other writers', per patch set. (b) Gemma's self-judging gap on each patch set, with 95% intervals. The open circle leaves out every held-out patch that needed a re-ask. The diamond is the pooled gap of five other models judging their own patches.

3.1 Gemma rubber-stamps its own failing patches

Patch set	Own failing patches scored 100	Others' failing patches scored 100	Leniency, own minus others' failing [95%]
Mini issues, mixed harness	61% (49 of 80)	20% (21 of 103)	+20.9 [11.5, 30.2]
Mini issues, matched harness	57% (12 of 21)	20% (21 of 103)	+11.7 [−1.4, 24.2]
Held-out issues	61% (52 of 85)	35% (136 of 393)	+16.6 [10.3, 23.1]

Gemma's rubber-stamp rate stays near 60% whatever the issues or the harness. It gives 100 to only 20 to 35% of other writers' failing patches. Some outside judges also give Gemma's failing patches 100 more often than other writers', so part of this difference may come from the patches. Leniency removes it by comparing judges on the same patch.

On the held-out issues, Gemma scores its own failing patches 14.1 points above the outside judges and other writers' failing patches 2.6 points below them. On passing patches, the difference is only +3.4 [−2.0, 8.0]. The held-out leniency is still +11.1 [3.8, 18.2] without the patches that needed a re-ask. On the 21 matched-harness failing patches the interval includes zero.

Gemma rarely sees the defect that outside judges see. On matplotlib-24570, it scores its own failing patch 100, calling it "a simple swap" that "correctly implements the expected behavior". GLM-4.5 scores the same patch 20 because it "breaks VPacker by reversing left and right", and the hidden tests for left and right alignment fail. We coded Gemma's answers on the 31 own failing held-out patches that it scored 100 and at least two outside judges scored 50 or less (exploratory). Two coders, both Claude models working apart, agree that Gemma never mentions those judges' main defect in 23 of them and names it in 3. They disagree on the other 5 (Supplement, Section D).

Thinking does not stop the rubber-stamping. Chen et al. (2025) find that reasoning before a verdict substantially reduces how often a judge prefers its own wrong answer to another model's correct one, in pairwise comparisons on math and knowledge questions. Our main runs switch thinking off, although every judge writes a short explanation before scoring. We scored the mixed-harness Mini patches again with Gemma's thinking on. Gemma still gives 100 to 62% of its own failing patches (61% with thinking off), and its AUROC on its own patches stays at 0.64. Thinking does lower its scores on the rest of its own failing patches, from a mean of 34 to 14. Its leniency therefore falls to +11.7 [1.1, 22.3] points, about half the mixed-harness value in the table above. A self-preference measure that averages scores can fall while the self-check gets no better.

3.2 Breaking the tie at 100 picks better patches to submit

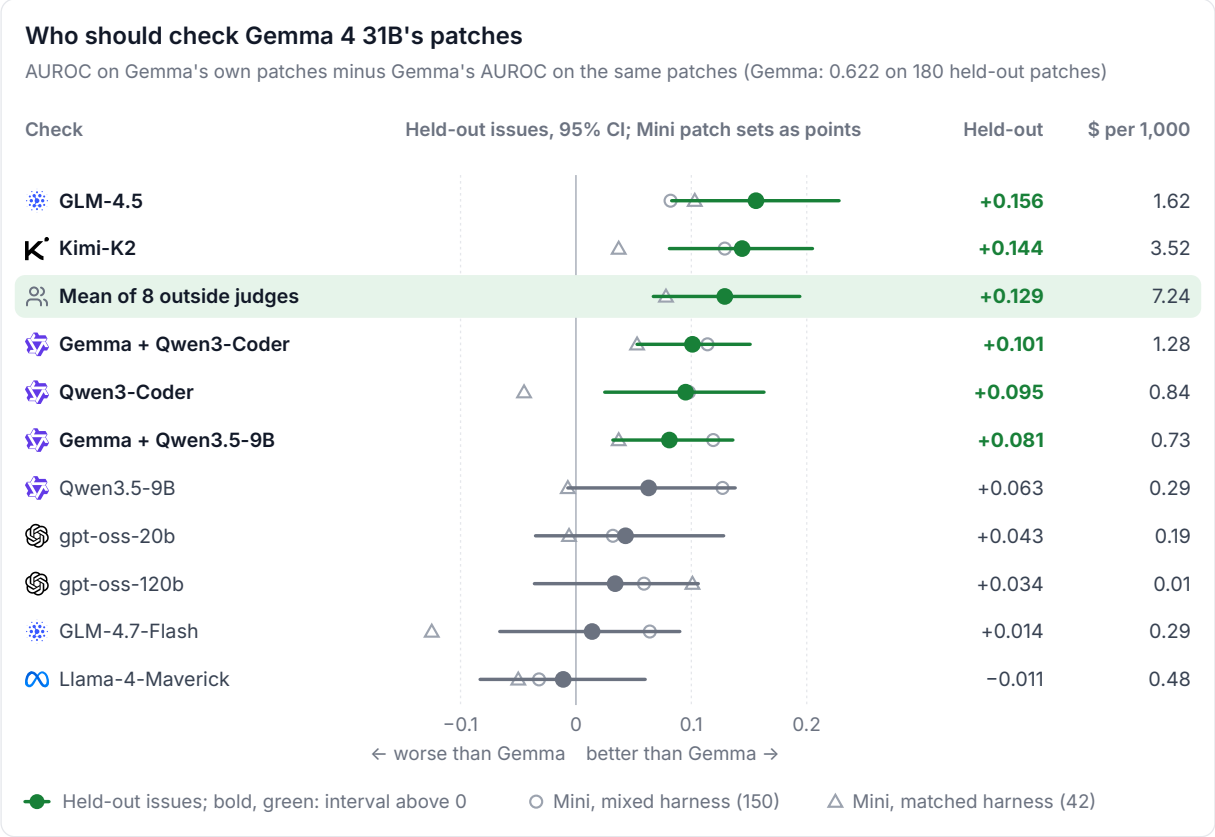


Figure 2. Each check's AUROC on Gemma's own patches minus Gemma's AUROC on the same patches. Lines are 95% intervals on the 180 held-out patches; hollow points are the two Mini patch sets. "Gemma + X" averages the two judges' ranks of each patch. Cost is what OpenRouter charged per 1,000 patches, in US dollars (gpt-oss-120b's provider reported almost zero cost).

We look at an agent that submits some of its patches without review and sends the rest to a person. The table below shows the share of held-out patches that pass when the agent submits the quarter or half that its check ranks highest. Without any check, 53% pass (95 of 180).

Check	Submit top quarter (45 patches): share passing, gain over Gemma's score in points [95%]	Submit top half (90 patches): same
Gemma's own score	61%	61%
Gemma, ties broken by fewer model calls	78%, +16.9 [5.7, 26.8]	72%, +10.8 [4.2, 16.5]
Gemma + Qwen3.5-9B	76%, +14.9 [6.7, 23.3]	68%, +6.6 [1.5, 12.0]
Kimi-K2	77%, +16.1 [9.6, 22.5]	72%, +11.5 [5.9, 17.6]
GLM-4.5	84%, +22.7 [12.7, 32.0]	74%, +12.6 [6.8, 18.3]

Gemma's score alone cannot lift the share passing above 61%. Gemma gives 100 to 133 of its 180 patches, and 81 of those pass (61%, by chance the same value as its rubber-stamp rate). Its top quarter and its top half are both random draws from that tied group. A second judge breaks the tie. GLM-4.5 and Kimi-K2 rank Gemma's patches better than Gemma does, by 0.16 and 0.14 in AUROC (Figure 2). Qwen3.5-9B alone is within the margin of error. Its average with Gemma's ranks gives a clear gain, at two cheap calls per patch.

We chose the three second-judge checks after seeing their AUROCs, so this comparison is exploratory. Over all 16 checks we ran, each outside judge alone and averaged with Gemma, the median top quarter passes 73% (Supplement, Section H). We also cannot tell whether the outside judges help because they come from another family or because they judge better in general.

Gemma's own agent run also breaks the tie, with no extra call. Among the patches Gemma scores 100, those whose run took fewer model calls pass more often. We chose this tie-break on the Mini issues, from three candidates, before testing it on the held-out issues, where it lifts the top quarter to 78%. This is within the margin of error of GLM-4.5, whose top quarter passes 5.9 points more [−3.6, 16.4] (exploratory). Run length may partly stand for how hard the issue is (Supplement, Section I).

3.3 Gemma ranks its own patches a little less accurately

Patch set	Gemma's AUROC, own patches	Gemma's AUROC, other writers' patches	Outside judges' drop	Self-judging gap [95%]
Mini issues, mixed harness	0.636	0.833	0.063	+0.135 [0.060, 0.210]
Mini issues, matched harness	0.696	0.833	0.071	+0.066 [−0.025, 0.161]
Held-out issues	0.622	0.751	0.055	+0.074 [0.022, 0.126]

Rubber-stamping should also hurt Gemma's ranking of its own patches, since a tie at 100 hides which of them fail. On the same 50 issues, the gap was about half as large when Gemma's patches came from the matched harness, though the two intervals overlap. The outside judges' drop stayed about the same, so the smaller gap comes from Gemma's own drop.

On the held-out issues, the gap meets the rule we fixed in advance for a replication (lower bound above zero and at least half the first estimate), but with little margin. It also depends on the larger re-ask limit. The gap is +0.030 [−0.024, 0.083] without the patches that needed a re-ask.

3.4 Five other models show no ranking gap and little leniency

On the held-out issues, all nine judges also scored the five other writers' patches, so each of these models judged its own patches, written in the same harness as Gemma's. Each wrote 46 to 123 failing patches. Their pooled gap is −0.001 [−0.031, 0.029] (Figure 1b), and their pooled leniency is +0.4 [−2.5, 3.4]. Gemma's leniency of +16.6 exceeds theirs by 16.3 [8.9, 23.4] points. No single model shows a gap or leniency once we correct for testing five models. This analysis is exploratory (Supplement, Section E, which also gives the Mini issues).

4 Auditing an agent's self-check

A team that wants to trust its agent's verdicts can run three steps on patches with known outcomes. The first compares the model with outside judges on the same patches, which accounts for how hard the patches are. The second matches the harness, because patches from different harnesses differ in more than their author. The third measures leniency on failing patches, patch by patch, because AUROC mixes leniency with issue difficulty and ties.

Some patches go out without review, such as submissions and pull requests. For an agent built on Gemma, we advise ranking these with a second model, or at least breaking Gemma's ties by fewer model calls. Our data cannot tell whether a second judge also helps choose among attempts at one issue (Supplement, Section D).

5 Related work

Judges that favor their own outputs have been studied mostly on text and short programs. Panickssery et al. (2024) link the strength of this self-preference to how well a model recognizes its own summaries, and Wataoka et al. (2024) relate it to the perplexity of the preferred response. Roytburg et al. (2026) compare a judge's own failing answers with other models' failing answers to the same question, and much of the measured self-preference goes away. Our leniency measure also compares judges on the same patch. Chen et al. (2025) find that stronger judges more often prefer their own wrong answer to a correct one.

Two studies are closest to ours. Pombal et al. (2026) check programs from twelve models against unit-test rubrics and find that judges favor their own outputs even with objective rubrics. On LiveCodeBench, Gemma 3 12B and 27B do not: their ratios are 0.97 and 1.03, where 1 means no self-preference. Khullar et al. (2026) find that a model rates a SWE-bench patch as more correct when it is framed as the model's own earlier answer. Ratings separate passing from failing patches with AUROC 0.99 when the patch is shown in a user turn instead. They report this for one unnamed model and do not compare it with other writers' patches. In our setting, the patches come from full agent runs, the judge sees no authorship or framing cue, and we measure ranking accuracy against hidden tests with outside judges as the control.

A second model that checks patches is already part of some SWE-bench agents. SWE-Gym (Pan et al., 2024) and R2E-Gym (Jain et al., 2025) fine-tune a verifier that reads an agent's run and patch and scores the chance that it passes, and they use it to choose one of several attempts at an issue. Our second judge is an off-the-shelf model, and we use it to decide which of an agent's patches go out without review.

6 Limitations

Our finding is about Gemma 4 31B on SWE-bench Verified. We do not know what causes rubber-stamping, or whether Gemma recognizes its own patches or shares the mistakes in them. Our checks on the mixed-harness patches find no support for two simple explanations, Gemma's smaller patches and the issues it cannot solve (Supplement, Section D). Our Gemma patches come from the 4-bit competition checkpoint.

Gemma as judge is the hosted model, and OpenRouter sent most of its calls to providers that serve 4-bit or 8-bit weights. We did not record the precision of each call.

Other writers' patches keep SWE-bench's official labels, and we graded our Gemma patches ourselves. On 30 issues, 26 other-writer patches got their official label in our environment. All four misses were official passes graded as failures, caused by environment faults and by patches that no longer apply to rebuilt images. If our grading made the same error on Gemma's patches, some of its failing patches would really pass, and a high score on them would look like rubber-stamping. We guarded against both causes: each issue we kept has a reference patch that passes in our environment, and Gemma wrote its patches on the same images. Our own runs made one attempt per issue, and the Mini issues give only 42 matched-harness patches, so the matched-harness rubber-stamp rate rests on 21 failing patches.

7 Data, code and supplement

All data are public, and no competition data was sent to an outside service. The repository (https://github.com/Vein05/gemma4-paper-track) holds Gemma's agent runs, every judge answer and the script behind every number. The supplement (`rubber-stamp-supplement.pdf`) holds the exact prompt, per-judge tables and the cost of each run (12.36 US dollars of API calls in all).

References

Wei-Lin Chen, Zhepei Wei, Xinyu Zhu, Shi Feng and Yu Meng. 2025. *Do LLM Evaluators Prefer Themselves for a Reason?* https://arxiv.org/abs/2504.03846.

Naman Jain, Jaskirat Singh, Manish Shetty, Liang Zheng, Koushik Sen and Ion Stoica. 2025. *R2E-Gym: Procedural Environments and Hybrid Verifiers for Scaling Open-Weights SWE Agents*. https://arxiv.org/abs/2504.07164.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press and Karthik Narasimhan. 2023. *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?* https://arxiv.org/abs/2310.06770. ICLR 2024 (arXiv comment, https://openreview.net/forum?id=VTF8yNQM66).

Dipika Khullar, Jack Hopkins, Rowan Wang and Fabien Roger. 2026. *Self-Attribution Bias: When AI Monitors Go Easy on Themselves*. https://arxiv.org/abs/2603.04582.

MariusHobbhahn. *swe-bench-verified-mini*. https://huggingface.co/datasets/MariusHobbhahn/swe-bench-verified-mini. Hugging Face dataset, revision b316c349, accessed 2026-10-05.

n-pelleriti. *AlphaDiana Gemma 4 agent-run datasets*. https://huggingface.co/n-pelleriti. Six Hugging Face datasets; accessed 2026-10-05.

Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, Heng Ji, Alane Suhr and Yizhe Zhang. 2024. *Training Software Engineering Agents and Verifiers with SWE-Gym*. https://arxiv.org/abs/2412.21139. Accepted at ICML 2025.

Arjun Panickssery, Samuel R. Bowman and Shi Feng. 2024. *LLM Evaluators Recognize and Favor Their Own Generations*. https://arxiv.org/abs/2404.13076.

José Pombal, Ricardo Rei and André F. T. Martins. 2026. *Self-Preference Bias in Rubric-Based Evaluation of Large Language Models*. https://arxiv.org/abs/2604.06996.

Dani Roytburg, Matthew Bozoukov, Matthew Nguyen, Jou Barzdukas, Mackenzie Puig-Hall and Narmeen Oozeer. 2026. *Are LLM Evaluators Really Narcissists? Sanity Checking Self-Preference Evaluations*. https://arxiv.org/abs/2601.22548. ICML 2026 Main.

SWE-agent. *mini-SWE-agent*. https://github.com/SWE-agent/mini-swe-agent. GitHub repository; accessed 2026-10-05.

SWE-bench. *SWE-bench experiments*. https://github.com/SWE-bench/experiments. GitHub repository, patches from the swe-bench-submissions bucket, revision 40f164d5, accessed 2026-10-05.

Koki Wataoka, Tsubasa Takahashi and Ryokan Ri. 2024. *Self-Preference Bias in LLM-as-a-Judge*. https://arxiv.org/abs/2410.21819. Accepted at NeurIPS 2024 Safe Generative AI Workshop.

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan and Ofir Press. 2024. *SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering*. https://arxiv.org/abs/2405.15793.